



Amrita School of AI
COIMBATORE CAMPUS

HIGH-PERFORMANCE AND CLOUD COMPUTING

Unit 0: Foundations

What a modern processor actually does, why it stopped getting faster, and how to measure a program honestly. Everything the rest of this course assumes you already know.

Prepared by **Dr. Abhijith Anandakrishnan**

Assistant Professor, Amrita School of AI

COURSE

23AID304 High-Performance and Cloud Computing

UNIT

0 — Foundations (preliminary)

READING TIME

About three hours

PREREQUISITE

C programming, first-year level

OFFERING

Odd Semester 2026-27

Before you start

This unit is not in the syllabus. It is here because the rest of the course does not work without it.

Units 1 to 4 will ask you to reason about cache lines, threads, memory bandwidth, and timing. Those words appear in second-year computer organisation courses and then get forgotten, because nothing forced you to use them. High-performance computing forces you to use them on day one.

So this unit rebuilds the foundation, from scratch, assuming only that you can write and compile a C program. If you already know it all, you will finish it in an hour and the rest of the course will be easier. If you do not, this is the unit that decides whether the rest makes sense.

BY THE END OF THIS UNIT YOU WILL BE ABLE TO

- Explain what a processor does in one clock cycle, and why more cycles per second stopped being the answer.
- Describe the memory hierarchy and put rough numbers on each level.
- Predict which of two loops will run faster, and say why, before running either.
- Distinguish a process from a thread, and say what each costs.
- Explain what a compiler does at each stage and what `-O2` changes.
- Time a program correctly, with the right clock, enough repetitions, and an honest statement of variance.
- Work comfortably enough on a Linux command line to use a cluster.

KEY TERMS IN THIS UNIT

clock cycle · instruction-level parallelism · pipeline ·
Dennard scaling · power wall · multicore · memory
hierarchy · cache line · spatial locality · temporal
locality · cache miss · latency · bandwidth · process ·

thread · address space · context switch · translation
unit · linking · wall-clock time · CPU time · monotonic
clock · warm-up · variance

What a processor actually does

The fetch-decode-execute loop

Strip away sixty years of engineering and a processor does one thing, forever:

1. **Fetch** the next instruction from memory, at the address in the program counter.
2. **Decode** it, to work out what operation it is and which operands it needs.
3. **Execute** it, in the arithmetic unit or the memory unit.
4. **Write back** the result, and advance the program counter.

This is the **von Neumann architecture**: instructions and data live in the same memory, and the processor reads both over the same path. Everything else in this course is a consequence of one fact about that design, which we will come to shortly.

DEFINITION · CLOCK CYCLE

The processor is driven by a square wave called the clock. One **cycle** is one tick of that wave, and it is the smallest unit of time the hardware recognises. A 3 GHz processor ticks three thousand million times a second, so one cycle is about 0.33 nanoseconds.

A useful mental anchor: **in one cycle, light travels about ten centimetres**. That is not a metaphor. It is why processors are small, why memory that sits on the other side of the motherboard is slow, and why a cluster spread across a room has an unavoidable communication cost.

Modern processors do far more than one instruction per cycle

The four-step loop above describes a processor from 1975. A processor from this decade overlaps the steps, in a technique called **pipelining**: while instruction n is executing, instruction $n+1$ is being decoded and instruction $n+2$ is being fetched. A pipeline fifteen stages deep can have fifteen instructions in flight at once.

On top of that, a modern core is **superscalar**: it has several arithmetic units and can start several independent instructions in the same cycle. It also executes **out of order**, reordering instructions to keep those units busy, and it **speculates** past branches, guessing which way an `if` will go and rolling back if it guessed wrong.

The practical consequence for you:

DEFINITION · INSTRUCTION-LEVEL PARALLELISM (ILP)

The parallelism the hardware finds *by itself*, within a single thread, by overlapping independent instructions. You get it for free, and you cannot control it directly. What you can do is write code that does not obstruct it: avoid long chains where every instruction needs the previous one's result, and avoid unpredictable branches in hot loops.

There is one more layer of free parallelism, and this one you will meet again in Unit 3:

DEFINITION · SIMD (SINGLE INSTRUCTION, MULTIPLE DATA)

One instruction applied to several values at once, using wide registers. An AVX-512 register holds eight doubles, so one `vaddpd` instruction performs eight additions. Compilers generate these automatically from simple loops, a process called **auto-vectorisation**. GPUs take this idea much further.

So a single core, running a single thread, is already executing several instructions per cycle, each of which may operate on eight numbers. Before you write a line of parallel code, the hardware is already doing a great deal in parallel. Your job in this course is the parallelism it *cannot* find on its own.

Why processors stopped getting faster

Until about 2005, you made programs faster by waiting. Clock speeds doubled every couple of years and yesterday's code ran twice as fast on today's chip. Then it stopped. Understanding *why* it stopped explains the entire existence of this course.

Moore's law is about transistor count, not speed

Moore's law is the observation that the number of transistors on a chip doubles roughly every two years. It says nothing about speed. That it *felt* like a speed law for forty years was due to a separate effect.

Dennard scaling, and its end

Dennard scaling observed that as transistors get smaller, their power density stays constant: smaller transistors need less voltage and less current, so you can pack more of them in the same area, run them faster, and keep the power budget the same. This is what turned more transistors into more speed.

Dennard scaling broke down around 2005. Below roughly 65 nanometres, the voltage could not keep falling: leakage current, the current that flows even when a transistor is nominally off, became significant, and threshold voltage could not scale further without the transistor failing to switch reliably.

The result is the **power wall**. Dynamic power in a CMOS circuit goes roughly as

$$P \approx C \cdot V^2 \cdot f$$

C capacitance · V supply voltage · f clock frequency

where C is capacitance, V is supply voltage and f is frequency. Once V stops falling, raising f raises power linearly, and in practice worse than linearly because higher frequencies need higher voltage to remain stable. Push a chip from 3 GHz to 6 GHz and you are looking at four to eight times the power in the same square centimetre. It melts.

COMMON MISCONCEPTION · CHIPS STOPPED GETTING FASTER BECAUSE WE RAN OUT OF PHYSICS

Not quite. We ran out of *cooling*. The transistors kept coming, exactly as Moore's law predicted. What ended was the ability to turn them into higher clock speed within a power budget you can remove as heat. So the industry spent the transistors on more cores instead of faster cores.

The consequence: parallelism is not optional

Since roughly 2005, performance has come from **more cores**, not faster ones. A 1990s programmer who ignored parallelism lost nothing. A programmer today who ignores it uses one core of ninety-six, and leaves 99 percent of the machine idle.

That is the whole argument for this course, in one sentence.

THE THREE WALLS

Power wall — cannot raise clock speed without melting the chip. **Memory wall** — processors got much faster than memory, so data movement now dominates. This is the next section. **ILP wall** — the

hardware has already extracted most of the parallelism available inside a single instruction stream; more transistors spent looking for it give diminishing returns.

The memory hierarchy

The central problem

Processor speed improved by roughly 50 percent a year for decades. DRAM latency improved by roughly 7 percent a year. The gap compounded, and today a main-memory access costs a few hundred cycles: a few hundred instructions the processor could have executed instead, but did not, because it was waiting.

This is the **memory wall**, and it is the single most important fact in practical high-performance computing. Most real programs are not limited by how fast the processor can compute. They are limited by how fast data arrives.

LEVEL	TYPICAL SIZE	LATENCY	FEELS LIKE
Registers on the core	~2 KB	0–1 cycles	1 second
L1 cache per core, split I/D	32–64 KB	~4 cycles	4 seconds
L2 cache per core	0.5–2 MB	~14 cycles	14 seconds
L3 cache shared by all cores	8–256 MB	~40 cycles	40 seconds
DRAM main memory	64 GB – 2 TB	~200 cycles	3 minutes
NVMe SSD local scratch	1–10 TB	~200 000 cycles	2.5 days
Network / parallel FS	100 TB +	~2 000 000 cycles	3 weeks

Figure 0.1 The memory hierarchy. Sizes and latencies are typical of a current server class processor; the exact figures vary by generation. The right-hand column rescales every latency as if one cycle took one second, which is the only way most people can feel the difference between four cycles and two hundred.

Read the right-hand column of Figure 0.1 again. If a register access felt like one second, going to main memory would feel like three minutes, and reading a file from the cluster’s shared filesystem would feel like three weeks. Programs spend most of their life waiting, and good HPC code is mostly the art of waiting less.

DEFINITION · LATENCY AND BANDWIDTH

Latency is how long one request takes: about 80 nanoseconds to DRAM. **Bandwidth** is how much data arrives per second once it is flowing: perhaps 200 GB/s on a server socket.

They are different problems with different fixes. Latency is hidden by having other work to do while you wait. Bandwidth is a hard ceiling: if your loop needs more bytes per second than the machine can deliver, no amount of cleverness in the arithmetic will help. Unit 1's roofline model makes this precise.

Caches, and the two kinds of locality

A **cache** is a small, fast memory that keeps recently used data close to the core. It works because real programs are not random. They exhibit locality:

DEFINITION · TEMPORAL AND SPATIAL LOCALITY

Temporal locality: if you used a value recently, you will probably use it again soon. A loop counter is the extreme case.

Spatial locality: if you used a value, you will probably use its neighbours soon. Walking an array is the extreme case.

Caches exploit temporal locality by *keeping* data, and spatial locality by fetching data in blocks rather than single bytes.

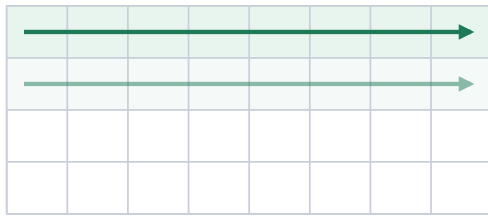
The cache line is the unit of everything

Memory does not move one byte at a time. It moves in fixed blocks called **cache lines**, almost always **64 bytes** on current hardware. That is eight double-precision numbers, or sixteen single-precision floats.

This one detail explains an enormous fraction of real-world performance.

Row order: j innermost

```
for i: for j: a[i][j]
```



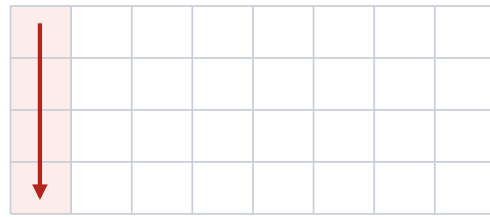
1 cache miss brings 8 useful doubles

Consecutive elements sit next to each other in memory. One 64-byte line fetch serves the next eight iterations, and the prefetcher sees the pattern and stays ahead of the loop.

~8 bytes of traffic per element

Column order: i innermost

```
for j: for i: a[i][j]
```



1 cache miss brings 1 useful double

Each step jumps a whole row. Every access pulls a fresh 64-byte line to use 8 bytes of it, and the line is usually evicted before the loop comes back for its neighbours.

~64 bytes of traffic per element: 8x worse

Figure 0.2 The same array, the same number of arithmetic operations, two loop orders. C stores a two-dimensional array row by row, so walking along a row uses every byte of each cache line, while walking down a column uses eight bytes of each 64-byte line and throws the rest away.

WORKED EXAMPLE · LOOP ORDER ON A 4096 BY 4096 MATRIX

Consider summing every element of a `double a[4096][4096]`, which is 128 MiB, far larger than any cache.

```
/* row order: j innermost */
for (i = 0; i < N; i++)
    for (j = 0; j < N; j++)
        s += a[i][j];
```

```
/* column order: i innermost */
for (j = 0; j < N; j++)
    for (i = 0; i < N; i++)
        s += a[i][j];
```

Both perform 16 777 216 additions. Both are correct. On a typical machine the row-order version runs **five to ten times faster**.

Why: in C, `a[i][j]` and `a[i][j+1]` are adjacent in memory. The row loop touches all eight doubles in each cache line before moving on, so it needs $16 \text{ Mi} \div 8 = 2 \text{ Mi}$ line fetches. The column loop jumps 4096 doubles, or 32 KB, between consecutive accesses, so every single access misses and needs its own line fetch: 16 Mi fetches, eight times the memory traffic. And because the stride is large and regular, the hardware prefetcher helps far less.

Nothing about the arithmetic changed. Only the order of the two loops did.

PITFALL · THIS IS THE MOST COMMON WAY UNDERGRADUATE HPC CODE IS SLOW

Before you reach for OpenMP, check your loop order. Parallelising a cache-hostile loop across eight cores gives you eight cores all waiting on memory. Fixing the loop order first is usually a larger win than parallelising, and the two multiply together.

Cache misses, and the three Cs

A **cache miss** is an access that was not found in cache and had to go to a slower level. Misses come in three flavours, and the classification tells you what to do:

KIND	CAUSE	WHAT HELPS
Compulsory	First ever touch of that data	Nothing: you must read it once. Prefetching hides the cost
Capacity	The working set is bigger than the cache	Blocking or tiling: restructure so the active data fits
Conflict	Several addresses map to the same cache set	Padding, changing array dimensions away from powers of two

Blocking, the fix for capacity misses, is the central optimisation in Unit 2's matrix work, and tiling with shared memory is the central optimisation in Unit 3's CUDA work. They are the same idea applied at different levels.

NUMA, briefly

On a machine with more than one processor socket, memory is physically attached to sockets. A core can read memory attached to its own socket faster than memory attached to the other one. This is **non-uniform memory access (NUMA)**.

The compute nodes of `asaicompute` are single-socket EPYC parts, but the head node is a two-socket machine, and EPYC processors are internally divided into groups with their own memory paths. The practical rule, which we return to in Unit 2, is **first touch**: on Linux, a page of memory is placed near whichever core first writes to it, not whichever core allocated it. Initialise your arrays with the same parallel loop that will later use them, and they land in the right place.

Processes and threads

Parallel programming means running several things at once. Operating systems offer two units for that, and confusing them is a reliable source of bugs.

DEFINITION · PROCESS

A running program with its **own private address space**. Two processes cannot see each other's memory. To communicate they must go through the operating system: pipes, sockets, shared-memory segments, or a message-passing library. Creating one is expensive, of the order of hundreds of microseconds.

DEFINITION · THREAD

An independent stream of execution **inside** a process. All threads of a process share the same address space, so they see the same variables and the same heap. Each has its own stack, its own registers, and its own program counter. Creating one is cheap, of the order of tens of microseconds.

The whole shape of this course follows from that distinction:

	THREADS	PROCESSES
Memory	Shared	Private
Communication	Read and write the same variables	Explicit messages
Main hazard	Data races	Deadlock, message mismatch
Scales to	One node	Many nodes
You will use	OpenMP (Unit 2)	MPI (Unit 2)

Shared memory is convenient and dangerous in exactly the same way: two threads can update the same variable without asking permission, which makes communication free and correctness hard. Message passing is verbose and safe in exactly the same way: nothing is shared, so nothing can be corrupted by accident, but you must say precisely who sends what to whom.

PITFALL · A THREAD IS NOT A CORE

You can create a thousand threads on a four-core machine. The operating system will time-slice them, and your program will run *slower* than with four, because every switch between threads costs a **context switch**: saving registers, reloading them, and usually throwing away the cache contents the old thread had warmed up. For compute-bound work, the right number of threads is the number of physical cores, not more.

BEYOND THE SYLLABUS · HYPERTHREADING, AND WHY HPC OFTEN TURNS IT OFF

Simultaneous multithreading (Intel calls it Hyper-Threading) presents one physical core as two logical ones, so a second thread can use the arithmetic units while the first waits on memory. It helps workloads with lots of stalls. It *hurts* dense numerical work, where a single thread already keeps the units busy and the two threads now fight over the same L1 cache. This is why `asaicompute` is configured with `ThreadsPerCore=1` in SLURM: the 96-thread nodes are scheduled as 96 cores, and the hyperthreads are deliberately ignored.

From source code to a running program

You have typed `gcc prog.c -o prog` many times. It is worth knowing what the four stages actually do, because in this course you will pass flags to each of them.

STAGE	TOOL	INPUT TO OUTPUT	WHAT IT DOES
Preprocess	<code>cpp</code>	<code>.c</code> to <code>.i</code>	Expands <code>#include</code> , <code>#define</code> , <code>#ifdef</code> . Pure text substitution
Compile	<code>cc1</code>	<code>.i</code> to <code>.s</code>	Parses, optimises, emits assembly. This is where <code>-O2</code> acts
Assemble	<code>as</code>	<code>.s</code> to <code>.o</code>	Turns assembly into machine code in an object file
Link	<code>ld</code>	<code>.o</code> + libraries to executable	Resolves symbols across files, attaches libraries

Two rules follow, and they explain most build errors you will hit:

- **Undefined reference** is a *linker* error. The compiler was happy: it saw a declaration in a header and believed you. The linker then could not find the definition. You forgot a `.c` file, or a library such as `-lm`.
- **Implicit declaration** is a *compiler* error. You used a function without declaring it, usually a missing `#include`.

Optimisation flags

FLAG	MEANING
<code>-O0</code>	No optimisation. Fast to compile, easy to debug, unusably slow. Never benchmark this
<code>-O2</code>	The sensible default. Inlining, loop optimisation, vectorisation
<code>-O3</code>	More aggressive, mainly extra vectorisation and loop transformation. Sometimes slower
<code>-g</code>	Emit debug symbols. Combines fine with <code>-O2</code>
<code>-march=native</code>	Use every instruction this specific processor supports. Faster; the binary may not run elsewhere
<code>-fopenmp</code>	Enable OpenMP. Without it, every <code>#pragma omp</code> is silently ignored
<code>-Wall -Wextra</code>	Turn on warnings. Use always

PITFALL · BENCHMARKING A DEBUG BUILD

Timing a program built with `-O0` and concluding your algorithm is slow is the most common error in this course. The gap between `-O0` and `-O2` is routinely a factor of three to ten. Always benchmark optimised builds.

PITFALL · THE OPTIMISER DELETED MY BENCHMARK

```
t0 = now();  
for (i = 0; i < N; i++) { double x = expensive(a[i]); } /* x unused */  
t1 = now();
```

The compiler can prove `x` is never used, so it removes the entire loop and you measure zero. Always *consume* the result: accumulate it into a variable and print it at the end, as `support/bench.c` does in the exercises.

Measuring time honestly

You cannot optimise what you cannot measure, and almost every beginner measures wrongly. There are four traps, and this course will fail your submissions for the first one, so read this section carefully.

Trap 1: the wrong clock

There are two completely different notions of “time taken”.

DEFINITION · WALL-CLOCK TIME AND CPU TIME

Wall-clock time is elapsed real time: what a stopwatch on the desk would show. This is what users care about, and it is what you must report.

CPU time is the total time processors spent executing your code. With eight threads running for one second of wall time, CPU time is about eight seconds.

The C standard function `clock()` returns **CPU time**. So:

```
clock_t t0 = clock();
#pragma omp parallel for
for (i = 0; i < n; i++) work(i);
double secs = (double)(clock() - t0) / CLOCKS_PER_SEC; /* WRONG */
```

Parallelise this perfectly across eight cores and `secs` goes *up* by a factor of about eight, making a perfect speedup look like an eightfold slowdown.

Use a wall-clock, monotonic timer instead:

```
/* OpenMP: the easy one, already wall-clock */
double t0 = omp_get_wtime();
...
double secs = omp_get_wtime() - t0;

/* Portable POSIX, no OpenMP needed */
struct timespec a, b;
clock_gettime(CLOCK_MONOTONIC, &a);
...
clock_gettime(CLOCK_MONOTONIC, &b);
double secs = (b.tv_sec - a.tv_sec) + 1e-9 * (b.tv_nsec - a.tv_nsec);
```

`CLOCK_MONOTONIC` is used rather than `CLOCK_REALTIME` because the latter can jump backwards when the system clock is adjusted, which produces negative durations.

Trap 2: measuring the first run

The first execution of anything is unrepresentative. Caches are cold, pages are not yet mapped, the OpenMP runtime has not yet created its thread pool, and the processor may still be at a low clock frequency. Always run a **warm-up** iteration and discard it.

Trap 3: measuring once

Timings vary. Another process wakes up, the scheduler migrates your thread to another core, the processor drops frequency because it got hot. One measurement tells you almost nothing.

Run the thing at least five times and report the **minimum**, not the mean. The minimum is the closest you got to running without interference; everything above it is noise added by the rest of the system. Report the spread too, so a reader can see whether the machine was quiet.

Trap 4: measuring something too short

If the work takes 200 nanoseconds and your clock has 100-nanosecond resolution, the measurement is meaningless. Repeat the work in an inner loop until the whole thing takes at least a few hundred milliseconds, then divide.

WORKED EXAMPLE · A TIMING HARNESS THAT IS ACTUALLY CORRECT

```
#include <time.h>
#include <stdio.h>

static double now(void) {
    struct timespec t;
    clock_gettime(CLOCK_MONOTONIC, &t);
    return t.tv_sec + 1e-9 * t.tv_nsec;
}

double time_best_of(void (*f)(void), int reps, int warmups) {
    for (int i = 0; i < warmups; i++) f();    /* discard: cold caches */
    double best = 1e300;
    for (int i = 0; i < reps; i++) {
        double t0 = now();
        f();
        double dt = now() - t0;
        if (dt < best) best = dt;            /* minimum, not mean */
    }
    return best;
}
```

Report it as: *"best of 5 runs after 1 warm-up, 0.842 s, spread 0.84 to 0.91 s, on one core of an EPYC 9654 at 2.4 GHz, gcc 13.2 with -O2 -march=native."* Anything less than that is not reproducible, and in this course a benchmark that cannot be reproduced does not count.

BEYOND THE SYLLABUS · REPRODUCIBLE BENCHMARKING IS A RESEARCH SKILL, NOT A COURSE RULE

Published performance claims are frequently unreproducible for exactly these reasons. When you report a speedup in your mini project, state the machine, the compiler and its flags, the problem size, the thread or rank count, the number of repetitions, and the statistic you report. This is the difference between a measurement and an anecdote.

A Linux survival kit

You will do everything in this course on Linux, most of it over SSH to `asaicompute`. This is the minimum that makes that comfortable.

Files and navigation

```
pwd                # where am I
ls -lh             # list, long form, human-readable sizes
cd /path/to/dir    # change directory; cd - returns to the previous one
mkdir -p a/b/c      # create nested directories, no error if they exist
cp -r src dst       # copy a directory
mv old new         # move or rename
rm -rf dir          # delete recursively. There is no undo. Read the path twice
du -sh *           # how big is each thing here
df -h .            # how full is this filesystem
```

Looking at files

```
less file.txt      # page through; q quits, /word searches
head -20 file      # first 20 lines
tail -f run.log    # follow a log as it is written: useful for a running job
grep -n "error" *.log # find lines, with line numbers
grep -rn "par_sum" . # search recursively
wc -l file         # count lines
```

Processes

```
htop               # interactive process viewer: watch your cores light up
ps aux | grep myprog
kill -9 <pid>      # last resort
nproc              # how many logical CPUs
lscpu              # full processor description, including cache sizes
free -h           # memory in use
```

`htop` is worth a minute of your time. Run your parallel program, watch the core meters, and see for yourself whether you are using one core or all of them. It catches more bugs than you would expect.

Redirection and pipes

```
./prog > out.txt           # stdout to a file
./prog 2> err.txt         # stderr to a file
./prog > all.txt 2>&1      # both to one file
./prog | grep SUMMARY     # feed output into another command
./prog | tee out.txt      # show on screen AND save
```

Remote work

```
ssh administrator@asai-11-admin    # log in
scp file.c user@host:~/           # copy a file up
scp -r user@host:~/results ./      # copy a directory down
rsync -avz local/ user@host:remote/ # copy only what changed. Prefer this
```

ON ASAI COMPUTE · USE TMUX OR SCREEN FOR ANYTHING LONG

If your SSH connection drops, every program you started dies with it. Run `tmux` first, work inside it, and press `Ctrl-b d` to detach. Reconnect later with `tmux attach` and your session is exactly as you left it. On a shared cluster this will save you at least once this semester.

Environment variables

```
export OMP_NUM_THREADS=8      # how many threads OpenMP should use
echo $OMP_NUM_THREADS
env | grep OMP
OMP_NUM_THREADS=4 ./prog      # set for one command only
```

`OMP_NUM_THREADS` is the single most important environment variable in this course. Forgetting it is why “my parallel code is not faster” is usually not a code problem at all.

What each unit will do with this

UNIT	USES FROM UNIT 0
1 Architecture and performance	Memory hierarchy and cache lines become the roofline model. Threads and processes become shared and distributed memory. Honest timing becomes speedup, efficiency, and scaling studies
2 OpenMP and MPI	Threads become OpenMP. Processes become MPI ranks. Cache lines come back as false sharing. Blocking fixes capacity misses
3 GPU and CUDA	SIMD widens into SIMT. The memory hierarchy gains registers, shared memory, and global memory. Cache lines become coalesced accesses. Tiling is blocking again
4 Cloud and HPC-AI	Latency and bandwidth reappear as network and storage limits. Processes and messages reappear as collective operations across GPUs

Almost nothing in this course is new. It is the same handful of ideas, applied at larger and larger scales.

Self-assessment

Work these before moving to Unit 1. Solutions follow, but the answers are worth much less than the attempt.

1. A processor runs at 2.4 GHz. How long is one clock cycle, in nanoseconds? If a main-memory access takes 80 ns, how many cycles is the processor stalled?
2. Explain in two sentences why manufacturers stopped increasing clock frequency, even though transistor counts kept rising.
3. A cache line is 64 bytes. How many `float` values (4 bytes) fit in one? How many `double` values?
4. You have `double a[1000][1000]`. Loop A sums it in row order, loop B in column order. Both compute the same total. Estimate the ratio of memory traffic between them, and explain the mechanism.
5. Give one reason to prefer threads over processes, and one reason to prefer processes over threads.
6. A student times an OpenMP program with `clock()`. Serial: 4.0 s. With 8 threads: 31.6 s. The program is correct and does scale well. What went wrong, and what would a correct measurement have shown?
7. Why report the minimum of several timing runs rather than the mean?
8. A program is compiled with `-O0` and takes 12 s. With `-O2` it takes 2 s. A classmate concludes the algorithm was improved eightfold. What is wrong with that conclusion?
9. Your loop reads one `double` from memory and performs one addition. The machine delivers 200 GB/s and can perform 3000 GFLOP/s. Which limit binds, and what performance do you expect? (This is the roofline model, arrived at early.)
10. Why does `asaicompute` schedule its 96-thread nodes as 96 CPUs rather than 192?

-
1. One cycle is $1 / 2.4 \text{ GHz} = 0.417 \text{ ns}$. An 80 ns access is $80 / 0.417 \approx 192$ cycles. On a superscalar core retiring up to four instructions per cycle, that is roughly 750 instruction slots lost to one memory access.
 2. Dynamic power scales as CV^2f , and once supply voltage stopped falling with transistor size (the end of Dennard scaling), raising frequency raised power and heat beyond what could be removed from the package. The extra transistors were therefore spent on more cores rather than faster ones.
 3. $64 / 4 = 16$ floats; $64 / 8 = 8$ doubles.
 4. Row order uses all 8 doubles per line, so it moves about 8 bytes of traffic per element. Column order strides 1000 doubles (8000 bytes) between accesses, so each access pulls a fresh 64-byte line and uses 8 bytes of it, giving about 64 bytes per element. Ratio is roughly 8 to 1, and the real gap is often larger because the hardware prefetcher

tracks the sequential pattern but not the strided one.

5. Threads: shared address space means communication is a plain memory access with no copying, and creation is far cheaper. Processes: isolated address spaces mean one process cannot corrupt another's memory, there are no data races, and the model extends across separate machines, which threads cannot do.

6. `clock()` returns CPU time summed over all threads, not elapsed time. With 8 threads the CPU time is roughly 8 times the wall time. A wall-clock measurement would have shown about $31.6 / 8 \approx 3.95$ s of CPU time corresponding to roughly **0.5 s of wall time**, that is a speedup near 8. The fix is `omp_get_wtime()` or `clock_gettime(CLOCK_MONOTONIC, ...)`.

7. Interference from the rest of the system can only ever make a run *slower*, never faster. The minimum is therefore the best estimate of the program's own cost, while the mean is contaminated by unrelated activity. Report the spread as well so the reader can judge how noisy the machine was.

8. Nothing about the algorithm changed: the same code was compiled with optimisation enabled. The comparison measures the compiler, not the algorithm. The `-00` figure is not a meaningful baseline; algorithmic comparisons must be made between optimised builds.

9. Arithmetic intensity is 1 flop per 8 bytes = 0.125 FLOP/byte. Bandwidth allows $200 \text{ GB/s} \times 0.125 \text{ FLOP/byte} = \mathbf{25 \text{ GFLOP/s}}$, far below the 3000 GFLOP/s compute ceiling. The loop is **memory bound**, and the machine will run at about 0.8 percent of peak arithmetic no matter how the code is written. Unit 1 formalises this.

10. The nodes have 96 physical cores with simultaneous multithreading giving 192 logical CPUs. Dense numerical work already saturates a core's arithmetic units, so a second thread on the same core mainly competes for the same L1 cache and slows both. Scheduling 96 CPUs gives each job whole physical cores and makes performance predictable and reproducible.

Where to go next

Unit 1 takes the memory hierarchy and the multicore argument from this unit and turns them into a quantitative theory: how to classify parallel machines, how to design a parallel algorithm, and how to say in advance how much faster it can possibly get. The roofline model in Unit 1 is question 9 above, done properly.